AI HALLUCINATION DETECTION STUDY

A Project Report on Identifying Unsupported or Incorrect Claims in AI-Generated Text

Project Area	Artificial Intelligence, Natural Language Processing, Information Retrieval
Project Type	Academic study / prototype design
Prepared for	Undergraduate project reference

This report explains the problem, concepts, system design, implementation plan, evaluation methods, limitations, and future scope of an AI hallucination detection system. It is a study template; measured results should be added only after the prototype is tested.

Contents

- 1. Abstract
- 2. Introduction
- 3. Problem Statement and Motivation
- 4. Aim and Objectives
- 5. Background and Key Concepts
- 6. Proposed System
- 7. System Architecture
- 8. Methodology and Workflow
- 9. Modules
- 10. Algorithms and Scoring
- 11. Technology Stack and Dataset
- 12. Evaluation Plan and Metrics
- 13. Expected Outcomes
- 14. Applications
- 15. Limitations and Ethical Considerations
- 16. Future Enhancements
- 17. Conclusion
- 18. Glossary
- 19. References and Further Reading

Suggested project scope: build a prototype that receives an AI-generated answer, breaks it into factual claims, checks those claims against trusted evidence where available, and reports which claims appear supported, contradicted, or unverifiable. The system should assist human review rather than claim to prove every statement true or false.

1. Abstract

Generative Artificial Intelligence (AI) systems can produce fluent, convincing text that contains incorrect facts, fabricated references, outdated information, or statements that are not supported by the available evidence. These errors are commonly called **AI hallucinations**. They can reduce user trust and create risks when AI output is used for education, research, customer support, healthcare information, or decision support.

The **AI Hallucination Detection Study** investigates how a software system can identify potentially unreliable claims in AI-generated text. The proposed prototype accepts an answer, divides it into smaller factual claims, retrieves relevant evidence from a selected knowledge source, compares each claim with that evidence, and assigns a label such as *Supported*, *Contradicted*, or *Unverified*. A confidence score and evidence snippets are shown to help a reviewer understand why a claim received its label.

The project combines natural language processing (NLP), information retrieval, sentence-level analysis, and optional natural language inference (NLI). Evaluation should measure claim-level precision, recall, F1-score, evidence quality, and the rate of unsupported claims that are missed. Since no detector can guarantee correctness in every domain, the system must communicate uncertainty clearly and retain a human-review step for important decisions.

2. Introduction

Large language models (LLMs) generate text by learning statistical patterns from training data and following an input prompt. They are useful for drafting, summarising, coding, tutoring, and question answering. However, fluent language is not the same as factual accuracy. A model may produce a plausible-sounding answer even when it lacks reliable information about a subject.

A hallucination can be a wrong date, an invented person or paper, a misquoted source, an incorrect numerical value, or a claim that sounds reasonable but cannot be supported by the evidence provided. Some errors are easy to check, while others depend on context, time, domain expertise, or the meaning of the question.

Hallucination detection attempts to flag such claims. A practical detector usually needs more than a language model that says whether another model is correct. It should seek external or supplied evidence, compare claims with that evidence, and distinguish between a claim that is contradicted and one for which there is simply not enough information.

2.1 Why this project matters

- Reliability: helps users identify claims that need verification before relying on AI output.
- Transparency: shows evidence and uncertainty instead of returning only a single opaque score.
- Education and research: encourages source checking and responsible use of AI-generated material.
- Quality assurance: supports human reviewers who need to triage long answers efficiently.

3. Problem Statement and Motivation

Problem statement: AI-generated answers may contain factual claims that are incorrect or unsupported. Users often cannot tell which parts of a fluent response are reliable. The project aims to design a system that identifies claim-level risk by comparing extracted claims with relevant evidence and clearly reporting the basis and uncertainty of its decisions.

3.1 Existing challenges

- **Fluent misinformation:** incorrect statements may be grammatically clear and confident.
- **Mixed accuracy:** one paragraph may contain both correct and incorrect claims.
- **Missing evidence:** a search may find no relevant source, which does not automatically prove the claim false.
- **Conflicting sources:** sources may disagree, be outdated, or have different levels of authority.
- **Context dependence:** a claim may be true only for a specific date, region, definition, or situation.
- **Evaluation difficulty:** a detector may incorrectly flag valid paraphrases or fail to catch subtle errors.

3.2 Proposed solution

Create a claim-based verification pipeline. The system extracts individual factual claims, retrieves relevant evidence from a controlled collection or search index, checks whether the evidence supports or contradicts each claim, and displays a label with citations or evidence excerpts. When evidence is weak or missing, the system returns *Unverified* rather than inventing a verdict.

4. Aim and Objectives

4.1 Aim

To study and prototype a method for detecting potentially hallucinated factual claims in AI-generated text using evidence retrieval and claim–evidence comparison.

4.2 Objectives

- Accept AI-generated text as input through a simple user interface.
- Split the response into sentences or atomic factual claims.
- Retrieve relevant evidence from a trusted document collection or approved web sources.
- Compare each claim with the retrieved evidence using rules, semantic similarity, or an NLI model.
- Classify claims as Supported, Contradicted, or Unverified.
- Show evidence excerpts, source details, and a confidence indicator.
- Evaluate the system against a labelled test set and report measurable performance.
- Document limitations, privacy considerations, and cases requiring human review.

4.3 Scope

A feasible student prototype can focus on English factual answers in one or two domains, such as general knowledge or course notes. It can use a small curated document set first, then expand to a search-based evidence source. The project does not need to prove every statement in the world; its goal is to identify risk and make verification more transparent.

5. Background and Key Concepts

5.1 AI hallucination

An AI hallucination is an output that presents information as factual even though it is false, fabricated, or not grounded in the relevant source or context. In retrieval-augmented systems, a related issue occurs when the answer goes beyond or conflicts with the retrieved documents.

5.2 Claim extraction

Claim extraction converts a long answer into small statements that can be checked separately. For example, the sentence “The library opened in 2012 and contains 40,000 books” contains at least two claims: (1) the library opened in 2012; (2) the library contains 40,000 books. Separating them allows the detector to label them differently.

5.3 Evidence retrieval

Evidence retrieval searches a knowledge base for passages relevant to each claim. A simple system can use keyword search such as BM25. A semantic search system can represent passages as embeddings and retrieve passages with similar meaning. A hybrid approach combines keyword and semantic results.

5.4 Natural language inference

Natural Language Inference (NLI) estimates the relationship between a premise and a hypothesis. In this project, a retrieved passage acts as the premise and the claim acts as the hypothesis. Common labels are entailment (evidence supports the claim), contradiction (evidence conflicts with the claim), and neutral (the evidence does not establish either). NLI output is a model estimate, not a guarantee.

5.5 Confidence and calibration

A confidence score expresses how strongly the system favours a label. A score of 0.90 should not automatically be described as a 90% probability of truth unless the model has been calibrated and tested for that interpretation. The user interface should explain that scores are estimates and can be wrong.

6. Proposed System

The proposed system uses a pipeline with six major stages: input, claim segmentation, evidence retrieval, evidence comparison, decision and reporting. A modular design allows each stage to be tested independently.

Stage	Input	Output
1. Text input	AI-generated answer	Cleaned text
2. Claim segmentation	Cleaned text	List of factual claims
3. Evidence retrieval	Claim + knowledge source	Ranked passages and source metadata
4. Claim–evidence comparison	Claim + passages	Support / contradiction / neutral estimates
5. Decision layer	Model scores and evidence quality	Final label and confidence indicator
6. Report interface	Labels, evidence, sources	Readable claim-level report

6.1 Functional requirements

- The user can paste text and submit it for analysis.
- The system can display each extracted claim separately.
- Each claim receives a label and an explanation or evidence snippet.
- The user can open the source used for verification where a source URL is available.
- The system can export or copy the report for review.

6.2 Non-functional requirements

- **Usability:** labels and explanations should be understandable to a non-expert.
- **Performance:** typical short answers should be processed within a practical time.
- **Traceability:** keep source identifiers and retrieval details for reproducibility.
- **Privacy:** avoid storing submitted text unless needed and clearly disclosed.
- **Robustness:** handle empty input, long text, missing sources, and retrieval errors.

7. System Architecture

A suitable high-level architecture is shown below. Each component can be implemented as a separate function or service.